

BRUTALTANK, MAPPED

Every box below reflects the code as it stands on the `master` branch (commit `469b92c`), not the aspirational shape in `PLAN.md` — the two have drifted in a couple of places, called out where it matters.

 Client module (Svelte/TS)

 Server module (Java)

 Plain domain/data class

 Broadcast event / flagged drift

 crosses the network (WS)

SYSTEM OVERVIEW

`client ↔ server`

Two independently-buildable modules, connected only by the JSON envelope contract in `shared/protocol.md`. Nothing else crosses the wire.

CLIENT — Svelte + TS, Vite :5173



{ type, v, requestId, payload }
shared/protocol.md — hand-maintained, no codegen

SERVER — Java + Undertow, :8080/ws

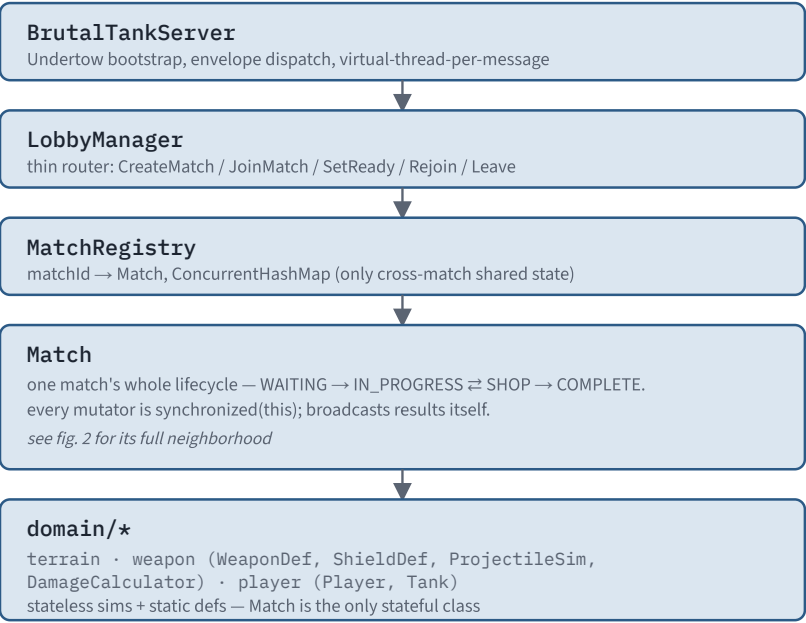
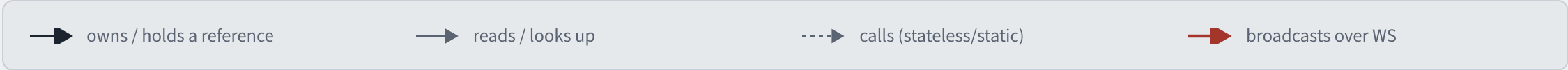


Fig. 1 — Both sides are plain layered stacks; the only coupling point is the WS envelope. The client never trusts its own angle/power as an outcome — it waits for ShotResolved.

SERVER DOMAIN MODEL

Match as hub

Everything server-side either *is* Match's state, or gets called by it. Arrow styles carry the relationship — see legend.



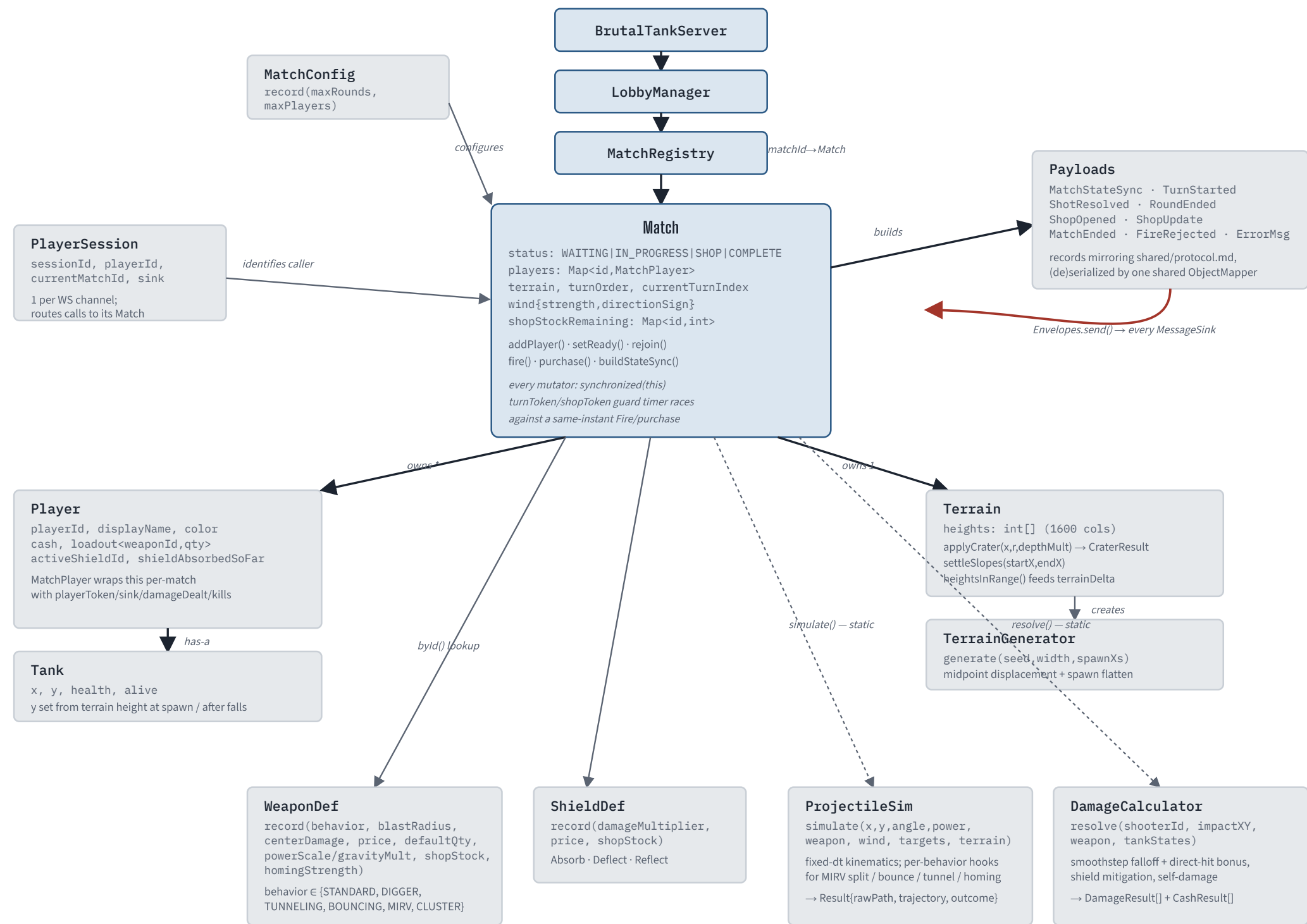


Fig. 2 — Match.java is 1,500 lines; everything else in domain/ is a small, mostly-stateless helper it calls into.

DRIFT PLAN.md §2.2 specifies a queue-based MatchActor (one thread, a command queue) as the concurrency backbone. The shipped code takes the simpler path Match.java's own javadoc documents: every mutator is synchronized(this), with a turnToken/shopToken counter to resolve the one real race — a 30s auto-skip timer firing at the same instant as a valid Fire. Same guarantee (a turn resolves exactly once), smaller mechanism.

CLIENT COMPONENT & STORE MAP

Svelte tree

One router, four screens, one canvas. Every store independently subscribes to the same wsClient message stream and reduces its own slice — there's no central action dispatcher.

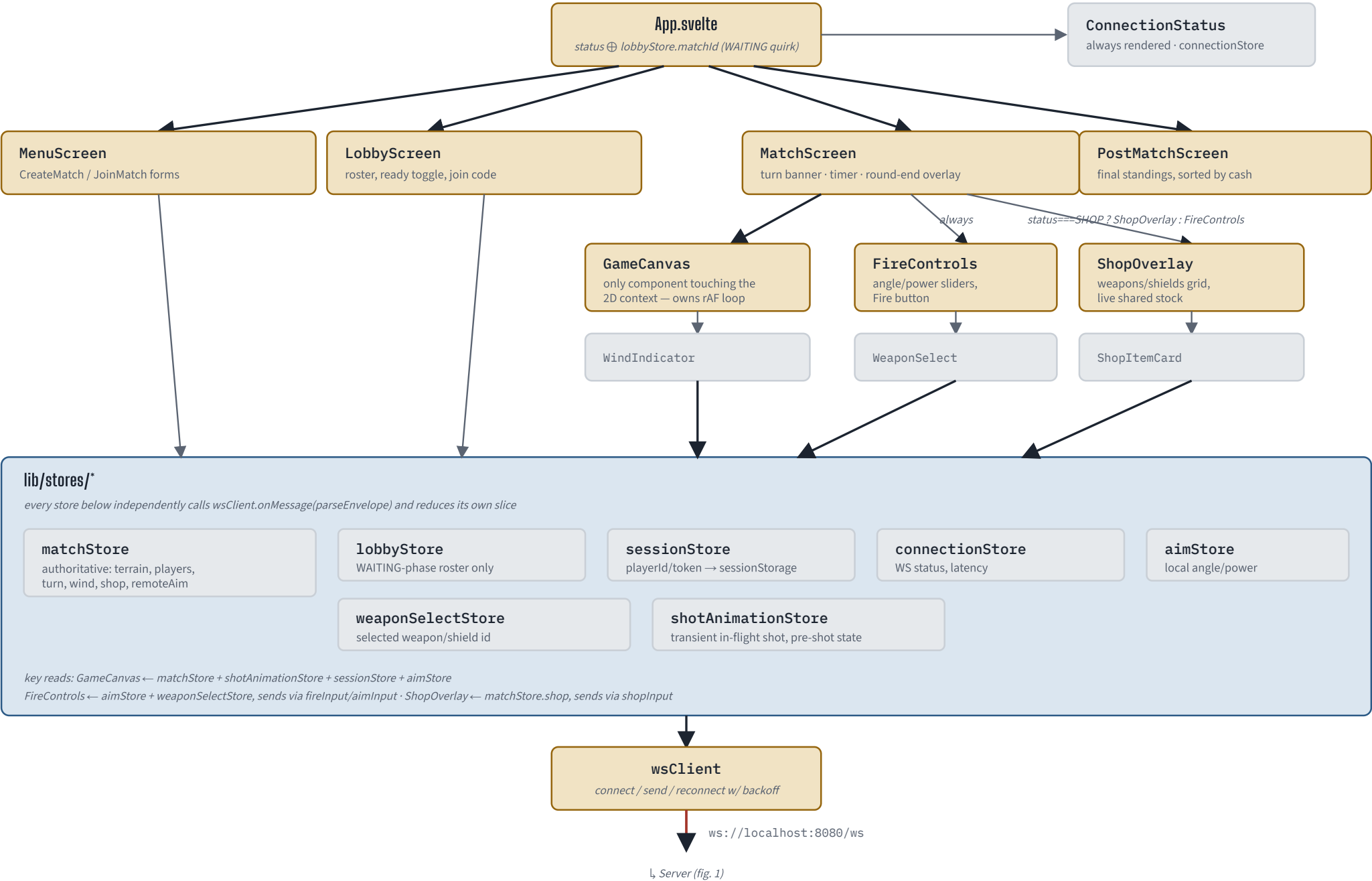


Fig. 3 — no store owns another; they're siblings that happen to agree on shape because they all read the same envelopes. The SHOP phase routes through MatchScreen, not a separate screen — an earlier bug sent it back to LobbyScreen instead, since lobbyStore.matchId is never cleared once a match starts.

MATCH LIFECYCLE & MESSAGE FLOW

the actual loop

What a player experiences, and the message that drives each transition. Two nested loops — turns inside a round, rounds inside a match — plus a disconnect path that can interrupt either.

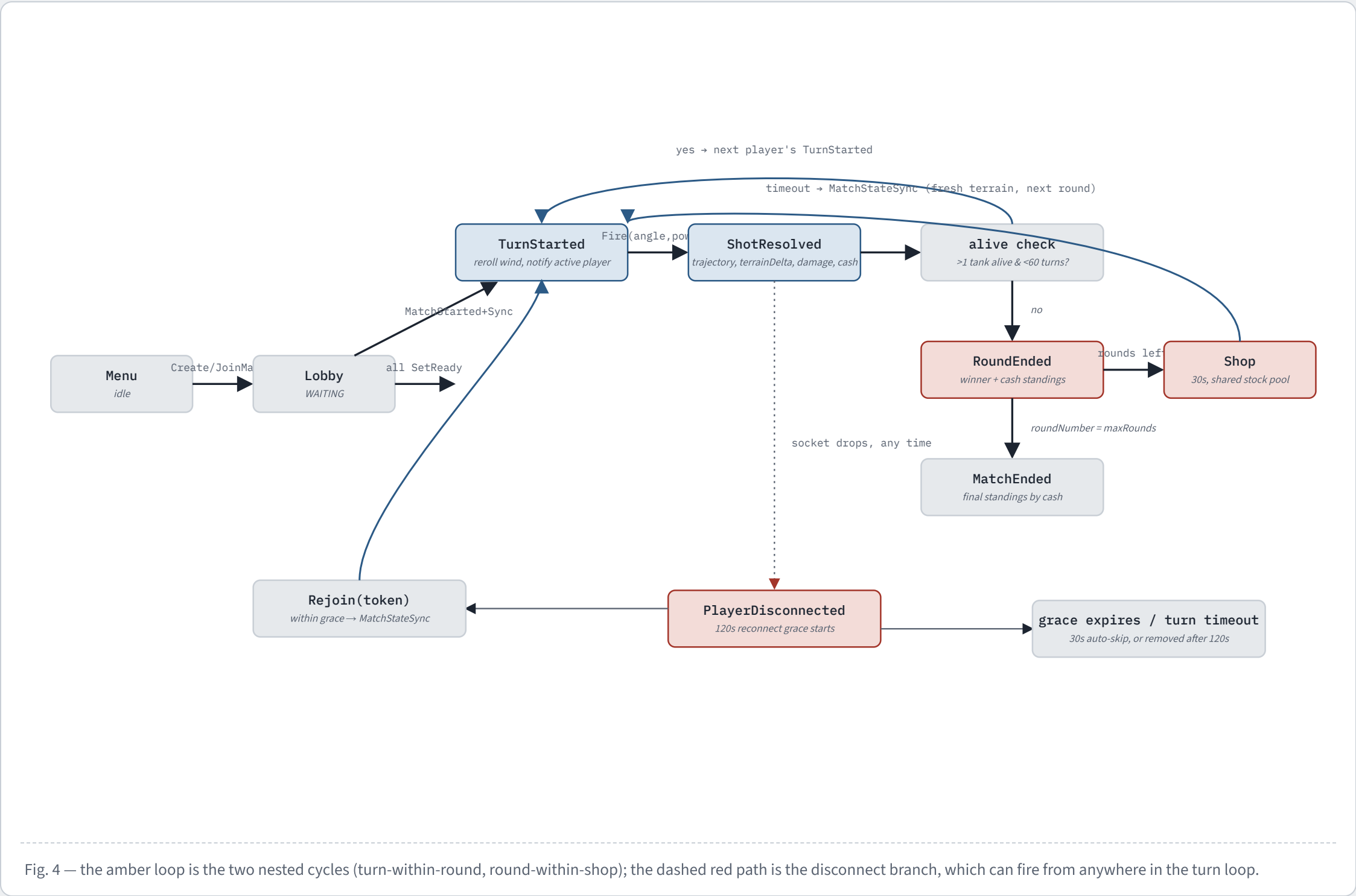


Fig. 4 — the amber loop is the two nested cycles (turn-within-round, round-within-shop); the dashed red path is the disconnect branch, which can fire from anywhere in the turn loop.

Ideas the user has raised and that are written up in `PLAN.md`'s "Future ideas" section, but not implemented — so they don't appear anywhere in the diagrams above.

- Single-player mode + bot opponent, driven through the same `Match.fire()` path a human uses
- Baby Missile-style terminal homing generalized (currently only `baby_missile` has nonzero `homingStrength`)
- Napalm damage-over-time / pooling in terrain cavities — needs genuinely new persistent turn-to-turn state
- Heavy Cannonball rolling downhill after impact, dealing further kinetic damage
- Bouncing Betty per-bounce damage (currently cosmetic skip-marks only)
- Digger real tunnel-then-collapse (currently reuses `TUNNELING` behavior, not yet playtested at its new size)
- Per-weapon distinct explosion art (only Nuke has one; everyone else shares a generic flash)
- Per-weapon sound design (research gathered in `docs/weapon-gap-analysis.md`, not built)
- Directional (trajectory-shaped) blast falloff instead of the current uniform radial falloff